Vilnius University

RESEARCH PROJECT

INTERNATIONAL CYBERSECURITY AND CYBERINTELLIGENCE

2026



VILNIUS UNIVERSITY
FACULTY OF MATHEMATICS AND INFORMATICS
INSTITUTE OF COMPUTER SCIENCE
CYBERSECURITY LABORATORY

International Cybersecurity and Cyberintelligence 1st year Research Project

Automated Threat-Intelligence Pipeline for First Response

Automatizuotas grėsmių analizės ir reagavimo sprendimas

Done by:
Emilija Baranauskaitė

Supervisor:
Teach. Asst. Virgilijus Krinickij

Vilnius
2026

Contents

Abstract **4**

Santrauka **5**

Introduction **6**

1 Related Work **7**
1.1 Detection Research and the Response Gap 7
1.2 Automated Incident Response 7
1.3 Choosing the Open-Source Tools 8

2 Methodology **8**
2.1 System Design 9
2.2 Attack Scenarios 11
2.3 Evaluation Methodology 12
2.4 Volumetric Analysis 13

3 Results **14**
3.1 Response-Time Breakdown 14
3.2 Comparison with Manual Response 15
3.3 Traffic Performance 15
3.4 False Alarm Analysis 16

4 Discussion **18**
4.1 Coverage limited to known attacks 18
4.2 Acting on a single signal 18
4.3 Throughput under high load 19
4.4 Securing the pipeline itself 19

Conclusions and Future Work **20**

References **21**

Abstract

Cyberattacks today are fast and often automated, but the response is usually still manual and slow, because analysts must read each alert, investigate it, and decide what to do. The majority of research on cybersecurity focuses on detecting attacks, while automatic response receives less attention. This work studies whether the first response to an attack can be fully automated, and how it compares with a manual one in speed and reliability. A pipeline was built from open-source components: Wazuh, a SIEM that detects the attack, and Ansible, an automation platform that applies the fix, joined by a short Python script that selects the response for each alert. It was tested against six attack scenarios based on the MITRE ATT&CK framework, with the full response time measured from a separate machine over one hundred experiments per scenario. Reliability was also checked: the stability of the response time on repeated attacks, the behaviour under many simultaneous alerts, and the reaction to harmless actions. In every experiment the pipeline responded successfully, taking under 10 seconds in five of the six scenarios and about 33 seconds for the SSH brute force, where the detection rule must first wait for many failed logins to build up. These times are far shorter than published figures for manual response, which range from minutes to days. The pipeline responded to every attack, and false alarms occurred only for harmless actions indistinguishable from an attack at the moment of detection. Overall, the work shows that, for attacks whose response is defined in advance, the first response can be fully automated with open-source tools and completed in seconds compared to manual response.

Santrauka

Automatizuotas grėsmių analizės ir reagavimo sprendimas

Šiuolaikinės kibernetinės atakos dėl jų automatizavimo yra tiek greitos, kiek ir dažnos, tačiau reagavimas į jas dažniausiai vis dar yra vykdomas rankiniu būdu. Taip yra dėl to, jog pagal nusistovėjusias reagavimo į kibernetines atakas praktikas analitikai turi įvertinti gaunamus įspėjimus ir priimti sprendimus dėl tinkamų atsako veiksmų. Panaši tendencija atsispindi ir šios srities mokslinėje literatūroje - dauguma mokslinių straipsnių yra orientuoti į atakų aptikimą, tuo tarpu automatizuotam incidentų suvaldymui skiriama gerokai mažiau dėmesio. Šiame tyrime nagrinėjama, ar pradinį atsaką į kibernetinę ataką galima visiškai automatizuoti, ir atitinkamai yra lyginama su rankiniu incidentų valdymu pagal reakcijos greitį bei stabilumą. Tyrimo metu buvo sukurta automatizuoto incidentų reagavimo sistema, paremta viešai pasiekiamo kodo (angl. open-source) komponentais: „Wazuh“ (SIEM sprendimas įvykiams aptikti) ir „Ansible“ (automatizavimo platforma atsakomosioms priemonėms vykdyti). Šie komponentai sujungti Python skriptu, kuris kiekvienam įspėjimui parenka tinkamą atsako veiksmą. Sistema buvo testuota šešiuose kibernetinių atakų scenarijuose, modeliuotuose pagal MITRE ATT&CK metodiką. Reakcijos laikas buvo matuojamas atliekant šimtą bandymų kiekvienam atakos scenarijui. Sistemos stabilumas buvo vertinamas analizuojant reakcijos laiko kitimą pakartotinių atakų metu, elgseną esant dideliam vienu metu generuojamų įspėjimų kiekiui bei reakcijas į neteisingai identifikuotus įvykius. Visuose testavimo scenarijuose sistema veikė sėkmingai. Penkiuose iš šešių atvejų reakcijos laikas siekė maždaug 6-10 sekundžių, o SSH *brute-force* scenarijuje - apie 33 sekundes, nes aptikimas buvo sukonfigūruotas taip, kad suveiktų tik sukaupus kelis nesėkmingus prisijungimo bandymus. Šis laikas yra ženkliai trumpesnis nei literatūroje nurodomas rankinio incidentų valdymo laikas, kuris gali siekti nuo kelių minučių iki kelių dienų. Sistema reagavo į visus bandymų įvykius, o neteisingai interpretuoti įspėjimai daugiausia kilo dėl įprastos vartotojo veiklos, kuri aptikimo etape nebuvo atskiriama nuo tikrų atakų. Tyrimo rezultatai leidžia daryti išvadą, jog tais atvejais, kai atsako veiksmai gali būti iš anksto apibrėžti, pradinis incidento valdymas gali būti visiškai automatizuotas naudojant atvirojo kodo įrankius ir įgyvendinamas per kelias sekundes.

Introduction

Cyberattacks are evolving rapidly alongside technological advances, becoming faster, more frequent, and more automated [6]. CrowdStrike [5] reports that in 2025 the average "breakout time" - the period between initial access and an attacker beginning lateral movement within a network - fell to just 29 minutes, down from 48 minutes the year before and roughly 70% lower than in 2021. In the fastest recorded case, a network was compromised in only 27 seconds.

Mandiant [13] reports a similar trend. Attacks are now commonly carried out by two specialised groups working in sequence: one that specialises in breaking in and a second that carries out the actual damage, such as a ransomware operator who buys the access. The median time for the first group to hand its access to the second has collapsed to just 22 seconds, down from more than eight hours in 2022. At the same time, the average time attackers go undetected has increased from 11 to 14 days, showing more hidden long-term attacks. In such conditions, manual incident response is often too slow. Analysts must review alerts, investigate activity, and decide on responses manually, which can take longer than the attack itself. As a result, even correct detection may not prevent damage. The importance of faster response is also shown by IBM Security [8], which reports that organisations using strong security automation reduced breach time by about 80 days and had lower costs than those relying on manual processes.

Despite this, much of the research focuses on threat detection [3], while automated response is less explored [11, 1, 12, 10, 9]. This creates a gap between identifying an attack and reacting to it in time. This research project addresses this gap by investigating whether a first-response process can be fully automated using open-source tools, and how this compares to manual response in terms of speed and reliability - that is, how consistently it responds to a repeated attack and how well it avoids acting on clearly benign activity. The hypothesis is that manual response typically takes minutes or hours, while automation can reduce this to seconds.

To evaluate this, an automated first-response system is developed and tested using six MITRE ATT&CK-based attack scenarios [24]. The main metric is total response time, measured from the first malicious activity to the completion of the response on the affected endpoint.

The rest of the paper is organised as follows. Section 1 reviews related work, Section 2 presents the system design and experimental setup, and Section 3 reports the results. Section 4 discusses the findings and limitations, while the conclusions summarise the study.

1. Related Work

This section reviews related work in two areas: how attacks are detected, and how the response to them is automated. The review shows that detection has been studied much more than response, and this is the gap that this project addresses. The section then introduces the open-source tools used to build the pipeline.

1.1. Detection Research and the Response Gap

Most published security research is about detecting attacks, with much less on automatically responding to them. This is clearly demonstrated in two review papers, both of which summarise findings from many studies. Smiliotopoulos et al. [18] examine 53 studies on lateral-movement detection, all concerned with how such activity is spotted and understood, not with how it is stopped. Srinivas et al. [19] screen 600 papers on the use of large language models (LLMs) in the security operations centre (SOC) and examine 105 in detail, reporting that most help analysts read and triage alerts while very few act on their own. Both surveys point in the same direction: the field has studied detection far more than response.

Other studies focus on the detection step itself, with the aim of making it faster or more accurate. Chamkar et al. [3] add machine-learning models to the alert pipeline of Wazuh, a widely used open-source SIEM, to reduce "alert fatigue," the flood of alerts that overwhelms analysts. This work improves how well alerts are produced, and the authors themselves point to automated response as the next step.

1.2. Automated Incident Response

Although detection dominates the literature, some studies do address the response step itself. A broad overview is given by Karlzén and Sommestad [11], who review 45 academic papers and four commercial tools. Their review shows that about two-thirds of the proposed responses are limited to network isolation or firewall changes, which leaves host-level remediation comparatively under-studied. It also notes that most proposed solutions are evaluated only conceptually or on small laboratory networks, rather than as running end-to-end systems.

Aljahdali and Alsulami [1] build an end-to-end SOAR system that automates common SOC tasks, such as brute-force handling and phishing, and report less work for analysts, though its actions stay mostly at the network edge, for example blocking an IP at the firewall. Looking at the playbooks themselves, Kremer et al. [12] present IC-SECURE, a tool for generating incident-response playbooks, and note that ordinary playbooks need heavy manual upkeep.

Another approach uses large language models to make the response more flexible. Ismail et al. [9] let an LLM select, run, and adapt response actions on its own, while in a related system the same authors [10] add an assistant to the Wazuh interface to help analysts interpret events. This flexibility is valuable, but it has a cost: to guard against wrong or invented actions, these systems rely on extra verification, including a human-interrupt step.

The closest prototype is by Thakker et al. [20], who show that automated host-level response is viable using Wazuh together with the Shuffle SOAR engine, routing alerts to a human through Microsoft Teams for optional intervention while still automating some responses. Of the work reviewed here, it is the most similar to the present project.

Farrel and Mardianto [7] address the same SSH brute-force attack handled in this project, using Wazuh's built-in active response to block the attacker's address automatically. Their setup differs in two ways that matter here: the block runs only on the host that detected the attack, so it protects a single machine, and they report detection and response as two separate phases rather than as one end-to-end time from attack to fix.

1.3. Choosing the Open-Source Tools

An automated first response needs two things: a tool to detect the attack, and a tool to carry out the response on its own. This project fills both roles with open-source software - Wazuh for detection and Ansible for response. It also uses Sigma, an open format that lets a detection rule be written once and then converted to many different security tools.

The detection tool chosen is **Wazuh**, an open-source SIEM that collects logs from monitored machines and raises an alert when its rules match. Wazuh fits this project for three concrete reasons. First, a single Wazuh agent provides all the host data the scenarios need, namely file-integrity monitoring, command monitoring, auditd command logging, and system-log collection, so no extra sensors are required. Second, it can forward an alert to an external program as soon as a rule fires, which is the hook the pipeline uses to start a response. Third, it runs as a central manager with lightweight agents on each endpoint, so one controller can watch many machines at once, which matches the setup tested here. A comparison of open-source SIEMs by Manzoor et al. [14] supports this choice, finding Wazuh fast and broad in endpoint coverage next to alternatives such as OSSIM, SIEMonster, and Elastic Security. Wazuh also offers an active-response feature that runs a script when an alert fires, but as Karlzén and Sommestad [11] note, it acts on only one machine at a time and relies on scripts the analyst must write, so acting on many machines at once is left to the user.

This limit is the reason for the second tool, **Ansible**. Ansible can make the same change on many machines at the same time, from one central computer. Red Hat guides [16] show that Ansible can block traffic, isolate machines, and apply patches after a security incident. This is the kind of action on many machines that Wazuh cannot do on its own, so Ansible is used to carry out the responses in this project's pipeline.

Alongside these two tools, the project uses **Sigma**, an open and vendor-neutral format for writing detection rules [17]. Sigma allows detection rules to be written once in a simple YAML format that is independent of any specific security tool. These rules can be automatically converted into different formats used by various security tools. This improves portability and removes the need to rewrite the same detection for each tool. While most detections in this project use Wazuh's native rule format, one rule is written in Sigma to show how a detection can be created in a tool-independent way instead of being written directly for Wazuh.

2. Methodology

This section explains how the system was built and how it was tested. It begins with the system design, covering the components and the four steps that every alert passes through, namely collection, detection, decision, and response (Section 2.1). It then presents the attack scenarios used for testing (Section 2.2), explains how the pipeline was evaluated (Section 2.3), and characterises the response time across the scenarios (Section 2.4).

2.1. System Design

The pipeline was evaluated in a test environment with five virtual machines running Ubuntu 22.04. The machines have three roles, shown in Figure 1. The controller is the central machine. It runs the Wazuh manager and the decision script, collects alerts from all endpoints, and starts the response. The three endpoints are the protected machines: a workstation, an Apache web server, and a database server. Each endpoint runs a Wazuh agent that sends monitoring data to the controller. The attacker is a separate machine on the same network, used only to perform test attacks against the endpoints.

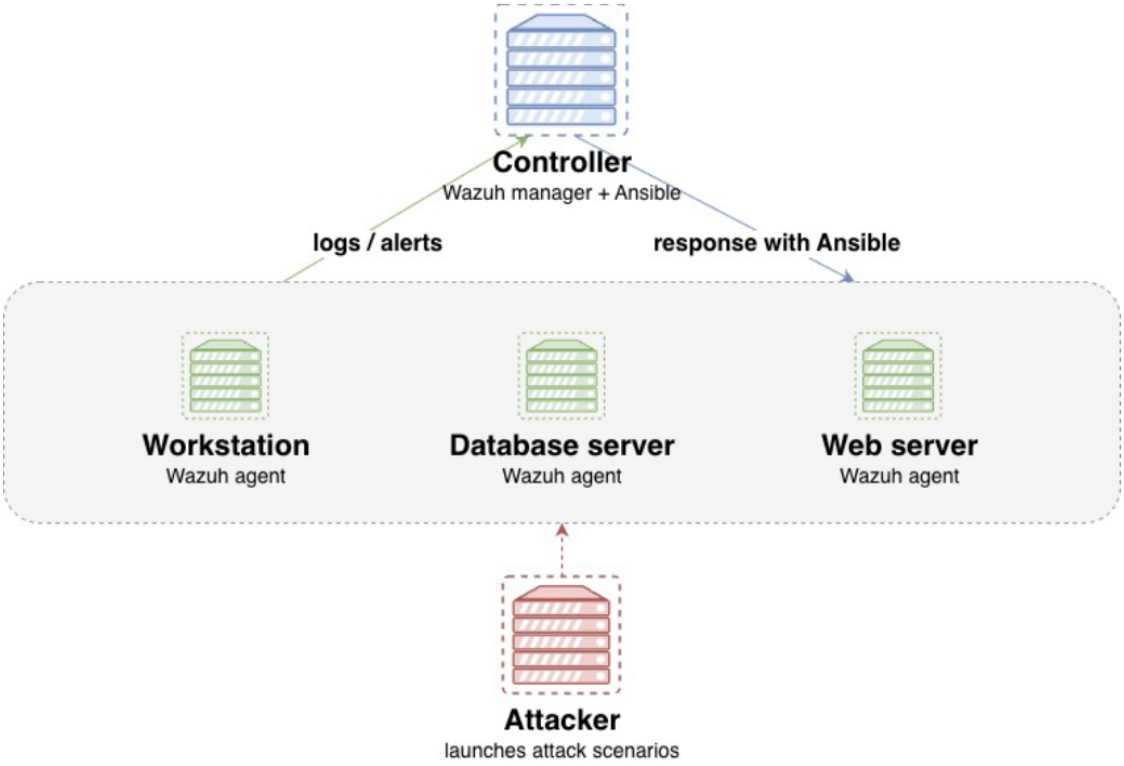


Figure 1: The laboratory environment and how the machines work together.

Figure 1 also shows how the machines work together. The attacker launches an attack against an endpoint, and the Wazuh agent on that endpoint sends logs and alerts to the controller. The controller analyses the alerts, decides on the response, and applies it to the endpoint through SSH using Ansible. Since all agents send their data to the same controller, it can monitor all endpoints and respond to incidents from one central location.

On each endpoint, Wazuh collects data from four sources. **File integrity monitoring (FIM)** detects changes to monitored files as soon as they occur. **Command monitoring** checks active network connections and running processes every five seconds. **Auditd** records the commands executed on the machine. In addition, Wazuh collects the standard **system logs** already generated by the endpoint, such as firewall and SSH logs.

An alert moves through the pipeline in one direction, with each stage passing its output to the next (Figure 2). It starts with **collection**, where the Wazuh agent on each endpoint gathers system logs, commands recorded by auditd, and changes to monitored files, and sends this data to the Wazuh manager on the controller. The manager then performs **detection**, where it compares the data against its rules. When a rule matches, the manager creates an alert in JSON format that includes the rule ID, endpoint information, and other relevant context.

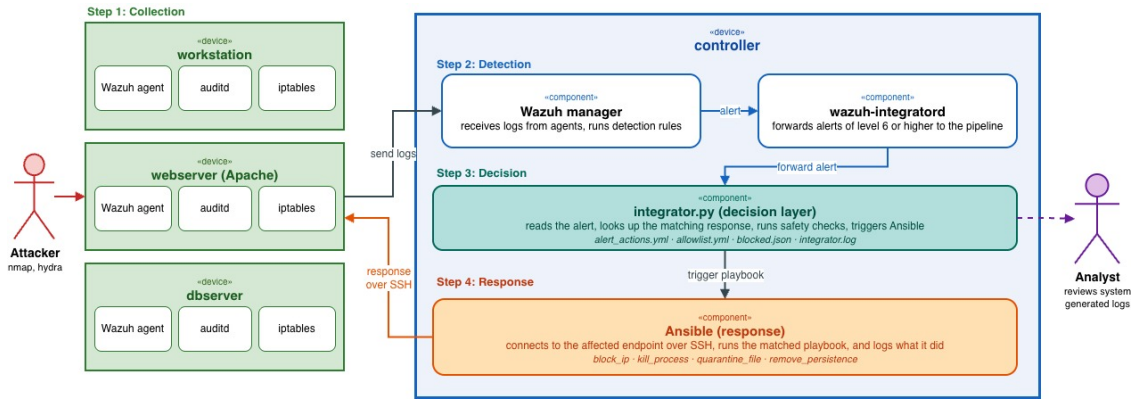


Figure 2: Overview of the pipeline flow: how an alert moves from collection to response.

Most alerts describe normal system activity and do not need any response. Wazuh assigns each alert a severity level from 0 to 15 [31], where levels 0 to 5 mean harmless events, such as a successful login. The `integratord` component forwards alerts to external programs only when the severity level is 6 or higher [30]. In this work, all response rules are set to level 10 or higher. This reduces noise but still allows all important alerts to pass. Alerts above this threshold are then sent to the decision script.

The decision is made by a Python script which reads the alert and looks up the corresponding response in a configuration file that maps each alert type to a specific playbook. It also extracts the required values from the alert, such as the attacker's address, the affected machine, or the path of a suspicious file. The script then executes the playbook with these values automatically.

Finally, the response is carried out by Ansible, which connects to the affected endpoint over SSH and applies the fix. The pipeline uses four playbooks, shown in Table 1 and explained below.

Table 1: The four response playbooks.

Playbook	Acts on	Response
`block_ip`	IP address	Drop all traffic from the address
`kill_process`	Process	Stop the malicious process
`quarantine_file`	File	Move the file to quarantine
`remove_persistence`	File	Delete the file, restart cron

The playbooks differ mainly in the action they perform. The **block_ip** playbook works at the network level by taking the attacker's address from the alert and adding an `iptables` rule that blocks all traffic to and from that address on every virtual machine. The other three playbooks operate at the host level on the endpoint where the attack is detected. The **kill_process** playbook stops the `bash` process that maintains an active network connection; stopping it can terminate a reverse shell. The **quarantine_file** and **remove_persistence** playbooks both use the file path from the alert field `syscheck.path`; **quarantine_file** moves the suspicious file into a quarantine directory for later analysis, while **remove_persistence** deletes the persistence file and restarts the cron service so that the attacker's scheduled task can no longer run.

Of the four playbooks, only **block_ip** acts on an IP address, so it includes two additional safety checks. First, the script checks the address against a list of trusted IPs to ensure that internal machines are never blocked. Second, it checks whether the same IP was already blocked in the last 24 hours to avoid repeating the same action. The block is executed only if both checks are passed.

The other three playbooks act on files or processes on the host, not on IP addresses, so these checks are not required for them. Algorithm 1 summarizes the full decision logic, from matching a rule to executing a playbook.

Algorithm 1 Decision logic for dispatching a response

Require: alert, ordered rule list, allowlist of trusted IPs
Ensure: one playbook runs, or no action is taken

```
for each rule in the list do
  if the rule matches the alert then
    extract the needed values from the alert (IP, host, file path)
    if the response is an IP block then
      if the IP is on the allowlist then
        log "allowlisted"; stop
      end if
      if the IP was blocked within 24 hours then
        log "already blocked"; stop
      end if
      run block_ip; record the IP as blocked
    else
      run the host playbook (kill, quarantine, or remove)
    end if
    log the outcome; stop
  end if
end for
```

Every action is written to a log in JSON format. Each line records the time, the rule that fired, the playbook that ran, the values it used, and the result. This log has two uses: it gives a clear record of what the system did, which an analyst can review afterwards, and it provides the timing data used to measure how long the response took.

2.2. Attack Scenarios

The system is evaluated using six attack scenarios. Each one was chosen because it is a common attack seen in recent reports, such as the 2025 Verizon Data Breach Investigations Report [28] and the 2025 Red Canary Threat Detection Report [15], and each is mapped to a technique in the MITRE ATT&CK framework [24]. Together, the scenarios cover different common steps in an attack, from scanning the network to trying to stay on the system. Each scenario is described in the same way: what the attacker does, how Wazuh detects it, and which playbook responds.

Port scan (T1046 [25]). The attacker uses a tool such as nmap to find open ports and decide where to strike next. Each endpoint logs every new connection with iptables, and a Wazuh rule counts them. The rule fires only on a burst, when one address makes 20 or more connections in 20 seconds. The pipeline then runs `block_ip` to drop that address on every endpoint.

SSH brute force (T1110.001 [21]). The attacker tries many passwords against the SSH service to guess a valid login, using the hydra tool. Wazuh already has built-in rules for repeated failed logins, so no custom rule is needed. When too many failures come from one address in a short time, the pipeline runs `block_ip` to block it.

Web shell upload (T1505.003 [27]). A web shell is a small malicious script placed on a web server so the attacker can control it through the browser. FIM watches the Apache web folder (`/var/www/html/`) and reports a new file as soon as it appears. The pipeline runs `quarantine_file`, which moves the file to a safe folder with a timestamp instead of deleting it, so it can be examined later.

Reverse shell (T1059.004 [22]). The endpoint opens a connection back to the attacker that hands over a command line. Because this is an ongoing connection rather than a single event, a check runs every five seconds and looks for a bash process that holds an active network connection. When it finds one, the pipeline runs `kill_process` to stop it.

Base64 execution (T1059.004 [22]). Attackers use base64 encoding to hide commands from simple detection filters. This scenario tests whether an encoded command that decodes and launches a reverse shell can still be detected in real time. The payload decodes a base64 string and starts a reverse shell as in the previous case; auditd logs the decode event immediately and a Sigma-based rule flags it. Unlike the plain reverse shell case, which uses periodic polling, this relies on event-driven detection. Both cases trigger the same response, `kill_process`, which terminates the active reverse shell connection rather than the encoded command itself. If no network connection is created, nothing is terminated.

Cron persistence (T1053.003 [26]). To stay on the machine after a reboot, the attacker adds a job in `/etc/cron.d/`, which Linux runs automatically. FIM watches this folder and reports a new file as soon as it appears. The pipeline runs `remove_persistence`, which deletes the file and restarts the cron service on that machine.

Across the six scenarios, detection covers files, processes, commands, and network and login logs. These scenarios exercise four distinct responses: port scan and SSH brute force both answer with `block_ip`, and reverse shell and base64 both answer with `kill_process`. Table 2 summarises, for each scenario, how it is detected and which playbook responds.

Table 2: How each attack is detected and which playbook responds.

Attack scenario	**Detected by**	**Response playbook**
Port scan	iptables connection log	`block_ip`
SSH brute force	Wazuh built-in SSH rules	`block_ip`
Web shell upload	FIM (file monitoring)	`quarantine_file`
Reverse shell	command check (every 5 s)	`kill_process`
Base64 execution	auditd command log	`kill_process`
Cron persistence	FIM (file monitoring)	`remove_persistence`

2.3. Evaluation Methodology

This section explains how the pipeline was evaluated. The evaluation has two main parts, speed and reliability. Speed is how long the pipeline takes to respond to an attack. Reliability is whether the pipeline reacts to real attacks and not to harmless activity. The pipeline is also tested under heavier load, to see how it behaves when many alerts arrive close together instead of one at a time.

Speed is measured with one value: the time from the start of an attack to the moment its fix takes effect on the affected machine. Wazuh detects the attack, the decision script chooses a response, and Ansible applies it. The time ends when the fix takes effect. For the two process-killing scenarios,

the reverse shell and the base64 execution, this is the moment the malicious process is gone. For the other scenarios, it is the log entry that confirms the playbook has finished. Each scenario is measured in one hundred experiments. The mean, standard deviation, minimum, and maximum are reported. The standard deviation shows how stable the response time is when the same attack is repeated.

An automated response is only safe if it acts on real attacks and not on normal activity, because a false alarm could block a real user or stop a legitimate process. To test this, the pipeline is given a set of harmless actions in addition to the attacks, and a correct pipeline takes no action on them. The harmless actions are split into three groups by how hard they are to tell apart from a real attack. Group A is clearly legitimate, such as a login with the correct password. Group B is close to a detection limit but stays below it, such as a small number of connections. Group C is identical to an attack at the moment of detection, such as a legitimate file added to the web directory. The pipeline passes when it does not react to Groups A and B, and when it reacts to Group C. For Group C, the pipeline cannot tell the harmless action from the attack, so a response is expected.

2.4. Volumetric Analysis

This subsection characterises the response time of each scenario. It provides a descriptive baseline of how the scenarios behave, which the analysis in Section 3 builds on.

Table 3: Response time of the six attack scenarios. 100 experiments per scenario; all values in seconds.

Scenario	mean	stddev	min	max
Web shell upload	5.79	0.39	5.00	6.99
Cron persistence	7.96	0.50	6.89	9.08
Reverse shell	9.81	1.88	6.20	12.93
Base64 execution	7.99	3.98	6.12	46.68
Port scan	7.01	0.80	5.48	9.06
SSH brute force	32.92	2.28	21.79	37.59

Table 3 reports the mean response time, its standard deviation, and the fastest and slowest values for each scenario. Each scenario was measured in one hundred experiments, six hundred in total. In every experiment, the pipeline detected the attack and applied its fix. The response time differs from one scenario to another, and the main reason is how each attack is detected.

The web shell is the fastest scenario, at about 5.8 seconds. File monitoring reports its file the moment it appears. The base64 command and the port scan come next, at about 7 seconds. The cron file takes about 8 seconds, where the higher total comes from the fix, which also restarts the cron service, and not from slower detection.

The reverse shell takes about 10 seconds. It is found by a periodic check that runs at a fixed interval of five seconds, rather than the instant the connection opens. Detection therefore waits for the next check, which adds a short and bounded delay.

The SSH brute force is by far the slowest, at about 33 seconds. Its rule fires only after many failed logins build up. Almost all of this time is spent in detection, waiting for that evidence, and not in the response. Section 3.1 confirms this.

The response time is also stable when the same attack is repeated. Across the one hundred experiments, the standard deviation in Table 3 stays below about twenty percent of the mean. The

reverse shell varies the most, because its check runs only every five seconds. Base64 is the one exception. Its higher mean and standard deviation come almost entirely from a single experiment, where detection took 43.8 seconds instead of the usual four. This was a temporary delay inside Wazuh, while the response stage stayed normal. The median time for base64 is about 7.5 seconds, in line with the other fast scenarios. The spike was caused by slower detection in that single experiment.

3. Results

This section evaluates the pipeline against the research question, using the response times characterised in Section 2.4. It first splits the response time into its stages, to show how fast the pipeline handles an attack (Section 3.1). It then compares this speed with manual response (Section 3.2). Next, it measures how the pipeline behaves under heavier load, when many alerts arrive close together (Section 3.3). Finally, it examines reliability, which is whether the pipeline responds to real attacks and not to harmless activity (Section 3.4).

3.1. Response-Time Breakdown

Table 4: Response time split into detection and response stages. Averages over 100 experiments per scenario, in seconds.

Scenario	detection	response	Ansible part	total
Web shell upload	1.25	4.53	3.60	5.79
Cron persistence	1.21	6.75	5.74	7.96
Port scan	2.36	4.66	4.01	7.01
Base64 execution	4.81	3.18	2.53	7.99
Reverse shell	6.28	3.54	2.53	9.81
SSH brute force	28.38	4.54	3.89	32.92

To see which stage takes the most time, every experiment recorded three moments: when the attack was launched, when Wazuh first raised a matching alert, and when the response playbook finished. The gap between the first two is the detection time; the gap between the last two is the response time, which is split further to show the part taken by Ansible. Table 4 gives the averages over the hundred experiments, ordered by detection time.

The clearest pattern is that detection, not response, drives how fast each scenario is. The response stage is steady across the board: Ansible applies the fix in about 2.5 to 4.0 seconds in five of the six scenarios, and only cron persistence takes longer, at 5.74 seconds, because its fix also restarts the cron service.

Detection times, in contrast, differ a lot from one scenario to the next. File monitoring reports the cron file in 1.21 seconds and the web shell in 1.25 seconds, and the iptables log reports the port scan in 2.36 seconds. The base64 command, which auditd records as it runs, is detected in 4.81 seconds on average - lifted slightly by the one slow experiment noted earlier, with a typical detection of about four seconds - a little faster than the reverse shell at 6.28 seconds, which has to wait for the next five-second check. The SSH brute force is the extreme case: about 86 percent of its total time (28.38 of 32.92 seconds) is spent waiting for enough failed logins to accumulate, while

its response stage, at 4.54 seconds, is no different from the rest. In short, how fast each scenario runs is set almost entirely by how quickly its detection method can recognise the attack.

3.2. Comparison with Manual Response

The goal of this pipeline is to respond fast, so it is useful to compare its response time with how these attacks are usually handled. This subsection puts the pipeline's measured times next to published times for that manual work, so the difference is easy to see.

The aim is to show the size of the difference, not an exact number, because no published study measures exactly the same thing as this work: the time from the attack to the fix being applied. Even so, the difference between the manual and the automated way is clear, because the pipeline finishes in seconds while the manual times are much higher. The Cloud Security Alliance study [2] reports that analysts needed on average 78 minutes to handle a brute-force alert and 105 minutes for a cloud-storage alert, and the IBM Cost of a Data Breach Report [8] reports an average of 60 days to contain a full breach.

The reason for this difference is simple: manual work is slow because it depends on people. The alert first waits in a queue, and only then does a person read it, investigate it, and decide what to do. Most of the time is spent waiting, not acting. The pipeline removes this waiting, because it only reacts to attacks whose response was decided in advance. It does not run a firewall command faster than a person could; it just never has to wait for a person to act. So for any attack with a planned response, automation is always faster, and the few seconds measured here are what is left once the human delay is gone.

3.3. Traffic Performance

The volumetric analysis in Section 2.4 measured one attack at a time. A real system is busier, and many alerts can arrive close together. This experiment measures how the pipeline performs as the number of alerts per second grows. The aim is to find the highest rate the pipeline can handle before responses start to wait. The rate is measured in *alerts per second*, not in network packets, because the pipeline acts on alerts. One alert can stand for a large amount of traffic, since a port scan of thousands of packets raises only one alert.

In this test, the alerts were sent straight to the response stage, without going through Wazuh detection. This lets the test set the exact rate at which alerts arrive. It also targets the right part of the pipeline, because the response stage handles alerts one at a time. Each response runs Ansible over SSH and takes a few seconds, so this stage is what limits how many alerts the pipeline can act on per second.

The requests were sent at rising rates, from 0.1 to 4 requests per second, twenty at each rate, each with a different address so the duplicate check did not suppress them. The time from submission to a completed block was recorded, and the results appear in Table 5 and Figure 3. Up to about 0.25 requests per second the pipeline keeps up, and each request finishes in about 3.6 seconds, the time to apply one block. From 0.5 requests per second the latency starts to rise, reaching 4.6 seconds, and at 1 request per second it reaches about 23 seconds, growing further after that. Above the rate the pipeline can serve, requests arrive faster than it can complete them, so they wait in a queue and each one takes longer.

Because the response stage handles one block at a time, its capacity is about one response every 3.6 seconds, or roughly 0.28 requests per second. Below this rate, the pipeline keeps up and the

Table 5: Pipeline performance at rising request rates. Twenty requests per rate; latency in seconds.

Arrival rate (req/s)	Completed	mean latency	p95	max
0.10	20/20	3.70	3.95	6.07
0.25	20/20	3.59	3.85	3.86
0.50	20/20	4.58	4.91	5.00
1.00	20/20	23.10	31.89	32.11
2.00	20/20	29.21	36.26	36.34
4.00	20/20	36.58	43.53	44.10

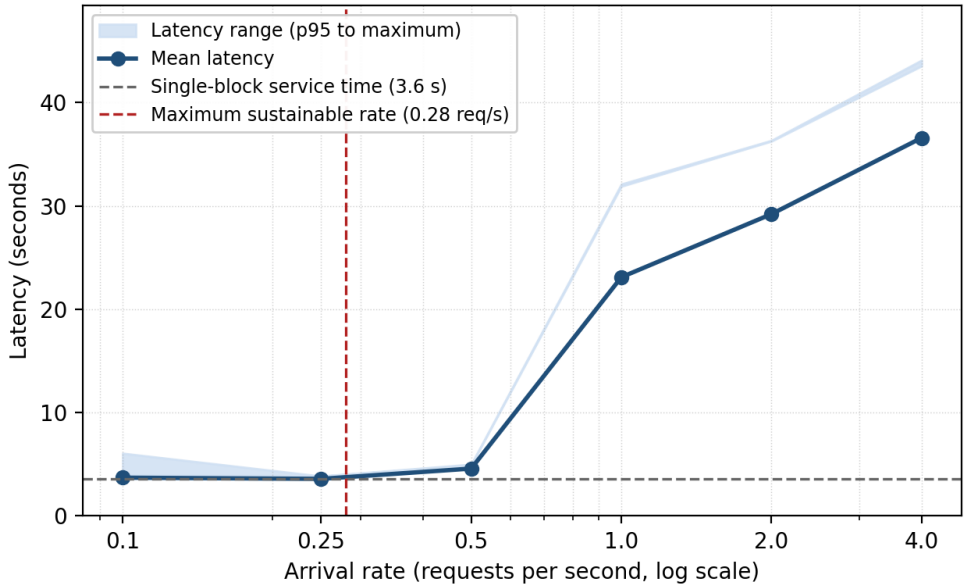


Figure 3: Response latency as the alert rate rises. It stays flat at low rates, then grows once requests start to queue.

latency stays flat. Above it, the queue grows and the latency rises. No request was ever lost, and every request completed at each rate, so under overload the pipeline becomes slower but does not drop work.

The same limit holds in a real environment. Wazuh sends each alert to the same decision script, so a burst of alerts is still handled one response at a time, and the last fix is delayed as the table shows. The test measured only the IP-block response, which must run one block at a time because all blocks share a single record of blocked addresses, so the measured limit applies specifically to it.

3.4. False Alarm Analysis

The previous sections measured the speed of the pipeline. This section measures its accuracy, meaning whether it responds to real attacks while leaving harmless activity untouched. This matters because a false alarm can block a legitimate user or stop a legitimate program. To measure it, the pipeline was given 120 labelled events, ten real attacks and ten harmless actions for each of the six scenarios, and its decision for each one was compared with the true label (Figure 4). The pipeline caught every attack, and its mistakes were false alarms, where harmless actions were treated as attacks.

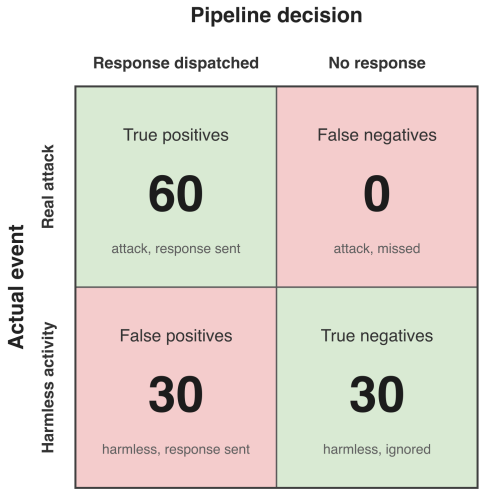


Figure 4: Confusion matrix for all 120 labelled events (60 attacks, 60 harmless actions).

All 30 false alarms came from three scenarios: web shell, cron persistence, and base64 execution. In each of these, the harmless action leaves the same trace as the attack, so the pipeline cannot distinguish the two and responds to both. A normal file in the web folder, a normal cron job, and a normal base64 command each look identical to their malicious version at the moment of detection. The other three scenarios produced no false alarms, because their harmless versions either stay below the alert threshold or do not match the rule.

To look at these cases more closely, eight harmless actions were tested and grouped by how easy they are to tell apart from an attack (Table 6). Group A is clearly safe, and Group B stays just below a detection limit, so the pipeline should ignore both. Group C, however, looks identical to an attack at the moment of detection, so here the pipeline is expected to react.

Table 6: The eight harmless-activity tests and their results.

Group	Test	Description	Expected	Observed
A	N1	SSH login with correct password	no-trigger	no-trigger
A	N5	Bash with no network use	no-trigger	no-trigger
A	N10	SSH port-forward (TCP owned by ssh, not bash)	no-trigger	no-trigger
B	N2	15 TCP connections in 20 s	no-trigger	no-trigger
B	N9	5 SSH connections in 20 s	no-trigger	no-trigger
C	N4	Legitimate base64 command	trigger	trigger
C	N7	Legitimate HTML in web root	trigger	trigger
C	N8	Legitimate cron job	trigger	trigger

As expected, Groups A and B caused no response. The boundary cases N2 and N9 stayed just below the connection limit. The clearest example is N10, an SSH port-forward: it carries network traffic like a reverse shell, but the rule reacts only when a `bash` process owns the connection, and here the connection belongs to the `ssh` client, so the pipeline correctly ignores it. The three Group C cases each caused a response: N7's file was quarantined and N8's cron entry was deleted. N4 was the exception, because the kill playbook acts only on a shell with a live connection, and the harmless base64 command opens none, so nothing was stopped and the system stayed unchanged.

The precision of 0.67 should not be read as the false-alarm rate of a real system. The test was made deliberately hard, because half of the harmless actions were chosen to be impossible to tell apart from an attack. In normal use, almost all activity looks nothing like an attack, so the real rate would be much lower. The low precision therefore does not show a weak detector. It shows the cost of acting on a single signal: when a harmless action leaves the same trace as an attack, the pipeline responds to both. Section 4 discusses this further.

4. Discussion

This section discusses the main limitations of the pipeline and the conditions under which each one becomes significant. Each limitation follows from how the system is built and from the results shown earlier, and each one leads to the future work set out in the conclusions.

4.1. Coverage limited to known attacks

By design, the pipeline separates detection from response: Wazuh provides the alerts and the pipeline acts on them. Its coverage therefore follows Wazuh's rules, each of which looks for a fixed sign of an attack. The pipeline responds to the patterns a rule was written for, while an attack that follows a new, unmatched pattern produces no automated response.

The response side mirrors this. The pipeline runs a playbook only where one is mapped to the alert, so an alert with no mapped response is logged for an operator to review rather than acted on automatically. The same holds when an attack is detected but has no response mapped to it.

Coverage can still be widened, because the design keeps detection and response separate. Wazuh does the detection with its rules, the playbooks carry out the response, and the pipeline connects the two. To add a new attack, one only writes a detection rule and links it to a playbook. The pipeline code itself does not change. The cost is that every new rule and playbook still has to be written and linked by hand, so more coverage means more manual work. One way to lower this cost is to write the detection rules in the portable Sigma format, as was already done for the base64 rule; the conclusions develop this idea further as future work.

4.2. Acting on a single signal

The pipeline makes each decision from a single piece of evidence, for example that a new file appeared or that a base64 command ran. The benign tests showed the effect (Section 3.4): a legitimate file in the web root was quarantined and a normal cron job was deleted, because each one matched its rule exactly. At the moment of detection, the pipeline cannot tell such legitimate actions apart from an attack that leaves the same sign. This is a known property of simple rule-based response [12].

This cost is acceptable for the setting the tool is made for. It runs on a monitored server where the watched locations, such as the web root and `/etc/cron.d/`, are expected to stay stable, so a file that appears there by itself is truly unusual. Any planned changes, such as software updates or maintenance, are handled by an operator, who can pause or adjust a rule during such work [4]. Telling benign activity and a real attack apart more reliably belongs at the detection stage, for instance by combining several signals, or by scoring alerts with a machine-learning filter before they reach the pipeline. Because detection and response are decoupled, such a filter sits in front of the pipeline without changing it, a direction the conclusions develop.

4.3. Throughput under high load

The pipeline carries out one response at a time. Each response applies its fix with Ansible and finishes before the next one starts, so responses form a queue when many alerts arrive together. The load test measured this limit (Section 3.3): the pipeline keeps up to about a quarter of a request per second, and above that rate each response waits longer.

This does not matter in the setting the tool was built for, where alerts that need a response happen from time to time rather than constantly. It would matter in a larger organisation, with many endpoints and frequent incidents, where response-worthy alerts could arrive faster than the pipeline can clear them. Even then no alert is lost, but the later fixes in a burst are delayed. Raising the limit means running several responses in parallel, which the conclusions note as future work.

4.4. Securing the pipeline itself

The pipeline protects the endpoints, but it does not protect itself. Like any central security tool, it puts a lot of trust in one place: the controller does the detection, keeps the rules, and can push a response to every endpoint at once. This makes the controller an important target. An attacker who takes control of it would not only switch off the defence, but also get a tool that already reaches every machine. The controller is therefore a single point of failure, and protecting it is as important as protecting the endpoints it watches.

Attacks on security tools are well known. Switching off or blinding such tools is a common step in many attacks, listed in MITRE ATT&CK as Impair Defenses (T1562), where attackers disable services that would show their activity [23].

There are known ways to defend against both. For the controller, Wazuh advises the same layered protection used for any sensitive server: keep it on a separate management network, keep the system updated, use access control such as SELinux or AppArmor, allow only key-based SSH login, and watch its own logs for signs of tampering [32]. For the agents, Wazuh has an anti-tampering feature that needs approval from the manager before an agent can be removed [29]. If an agent is force-stopped, the manager notices that its regular keep-alive messages have stopped and raises a disconnection alert, so the removal can be seen. Sending the logs to the controller as they are made gives one more safeguard: the logs leave the endpoint at once and are still there even if the machine is later wiped.

These protections were outside the scope of this prototype. The evaluation looked at response time and reliability, not at how well the pipeline holds up when it is attacked, and the lab controller used only default hardening.

Conclusions and Future Work

This work studied whether the first response to a common attack can be fully automated, and how fast such a response is compared with a human one. For this, a pipeline was built from open-source tools. It joins detection and response into one flow that picks and runs a fixed playbook for each alert (Section 2.1). The pipeline gave a full response in all trials, in a few seconds for most scenarios. A manual response usually takes much longer, from tens of minutes to days. So for attacks whose response is decided in advance, automation removes almost all of the delay that comes from waiting for a person.

The work also has clear limits (Section 4), and each one points to a next step. The first step is to scale the pipeline up. The evaluation used only a few machines, and the pipeline handles one response at a time, so in a larger organisation with many endpoints the alerts could arrive faster than it can act on them. Running the pipeline on a larger setup, and letting it process several responses in parallel, would let it handle this higher load.

A second step is to reduce the false positives of single-signal detection. Because the pipeline acts on one piece of evidence, a normal file or command that matches a rule is treated the same as a real attack. This could be lowered by checking a second piece of evidence before acting, or by filtering the alerts first so that only confirmed ones start a response. Chamkar et al. [3] show that this works in practice: a machine-learning model added to Wazuh lowers false alarms, so the response would run only on alerts that are already confirmed.

A further step is to make the response itself smarter with a large language model (LLM). In the current pipeline the response is fixed in two ways. First, the choice of action is a simple lookup: the script matches the alert against a table and runs the linked playbook, always the same one. An LLM could instead read the full alert and choose the response that fits it, or change the steps to match the details of the attack, so the response is shaped by the alert and not decided in advance [9]. Second, every playbook is still written by hand before the pipeline runs. The base64 rule is already written in the portable Sigma format, so an LLM could read such a rule and write the matching playbook on its own [12]. Together, these two changes would let the pipeline respond to new attacks without a person writing each playbook from scratch, and let every response adapt to the alert instead of staying fixed, which is the part of automation this work could not yet reach.

A final step is to widen the scope. The setup used only Ubuntu hosts, so testing on other systems such as Windows would show how far the design carries [20, 14]. Confirmed incidents could also be sent to a ticketing system, to fit a real operations centre [1].

Overall, this work shows that a first-response system can be built from existing open-source tools, and that for attacks whose response is known in advance, it removes almost all the waiting that a human response would add. The pipeline is still a small prototype, tested in a single lab and able to handle only the attacks that fixed rules can detect. The steps described above each address one of these limits, from a larger test setup and better filtering of false alarms to a model that helps choose and write the responses and support for more systems, moving the pipeline closer to real use.

References

[1] Asia Othman Aljahdali and Raghad Alsulami. Streamlining threat response and automating critical use cases with security orchestration, automation and response (soar). *Journal of Digital Security and Forensics*, 2(1):36–57, 2025.

[2] Hillary Baron and Kurt Seifried. Beyond the hype: A benchmark study of AI agents in the SOC. Research report, Cloud Security Alliance, Seattle, WA, USA, October 2025. Sponsored by Dropzone AI; 148 randomised participants; study conducted July-August 2025. https://cloudsecurityalliance.org/artifacts/a-benchmark-study-of-ai-agents-in-the-soc.

[3] Salah Eddine Chamkar, Mounia Zaydi, Yassine Maleh, and Nadia Gherabi. Improving threat detection in Wazuh using machine learning techniques. *Journal of Cybersecurity and Privacy*, 5(2), 2025. Article 34.

[4] Paul Cichonski, Tom Millar, Tim Grance, and Karen Scarfone. Computer security incident handling guide. NIST Special Publication 800-61 Revision 2, National Institute of Standards and Technology, Gaithersburg, MD, USA, August 2012.

[5] CrowdStrike, Inc. 2026 global threat report. Industry threat report, 2026. https://www.crowdstrike.com/global-threat-report/.

[6] ENISA. ENISA threat landscape 2025. European Union Agency for Cybersecurity, 2025. https://www.enisa.europa.eu/publications/enisa-threat-landscape-2025.

[7] Farhan Ibnu Farrel and Is Mardianto. Implementation of security information & event management (SIEM) Wazuh with active response and telegram notification for mitigating brute force attacks on the GT-I2TI USAKTI information system. *Intelmatics*, 4(1), 2024. https://e-journal.trisakti.ac.id/index.php/intelmatics/article/view/18529.

[8] IBM Security and Ponemon Institute. Cost of a data breach report 2025. Annual benchmark report, IBM Corporation, Armonk, NY, USA, July 2025. 20th annual edition; n = 600 breached organisations across 17 industries and 16 countries; 3,470 interviews. https://www.ibm.com/reports/data-breach.

[9] Rahmat Kurnia Ismail, Zilmas Arjuna Brata, Ghina Azmita Nelistiani, Seyoung Heo, Haengrok Kim, and Howon Kim. Toward robust security orchestration and automated response in security operations centers with a hyper-automation approach using agentic artificial intelligence. *Information*, 16(5), 2025. Article 365.

[10] Rahmat Kurnia Ismail, Farell Widyatama, I Made Wibawa, Zilmas Arjuna Brata, Ukasyah, Ghina Azmita Nelistiani, and Howon Kim. Enhancing security operations center: Wazuh security event response with retrieval-augmented-generation-driven copilot. *Sensors*, 25(3):870, 2025.

[11] Henrik Karlzén and Teodor Sommestad. Automatic incident response solutions: A review of proposed solutions' input and output. In *Proceedings of the 18th International Conference on Availability, Reliability and Security (ARES '23)*, pages 1–9. Association for Computing Machinery, 2023.

[12] Ryan Kremer, Prasasthy Narayanan Wudali, Satoru Momiyama, Toshinori Araki, Jun Furukawa, Yuval Elovici, and Asaf Shabtai. IC-SECURE: Intelligent system for assisting security experts in generating playbooks for automated incident response. arXiv preprint arXiv:2311.03825, 2023. https://arxiv.org/abs/2311.03825.

[13] Mandiant. M-Trends 2026 executive edition. Mandiant / Google Cloud, 2026. https://cloud.google.com/security/resources/m-trends.

[14] J. Manzoor, A. Waleed, A. F. Jamali, and A. Masood. Cybersecurity on a budget: Evaluating security and performance of open-source SIEM solutions for SMEs. *PLOS ONE*, 19(3):e0301183, 2024.

[15] Red Canary. 2025 threat detection report. Industry threat report, 2025. https://redcanary.com/threat-detection-report/.

[16] Red Hat. Ansible security automation: Incident response. Red Hat Blog, n.d. Retrieved 2026-05-28 from https://www.redhat.com/en/blog/getting-started-with-ansible-security-automation-incident-response.

[17] SigmaHQ. Sigma - generic signature format for SIEM systems. Project documentation, n.d. Retrieved 2026-05-28 from https://sigmahq.io/.

[18] C. Smiliotopoulos, G. Kambourakis, and C. Kolias. Detecting lateral movement: A systematic survey. *Heliyon*, 10(4):e26317, 2024.

[19] S. Srinivas, B. Kirk, J. Zendejas, M. Espino, M. Boskovich, A. Bari, K. Dajani, and N. Alzahrani. AI-augmented SOC: A survey of LLMs and agents for security automation. *Journal of Cybersecurity and Privacy*, 5(4), 2025. Article 95.

[20] A. Thakker, A. More, and K. Kumar. Automated defense against application-layer attacks on Windows systems using Wazuh and Shuffle. *International Journal of Education, Science, Technology, and Engineering*, 8(1), 2025.

[21] The MITRE Corporation. Brute force: Password guessing (T1110.001). MITRE ATT&CK, 2025. https://attack.mitre.org/techniques/T1110/001/.

[22] The MITRE Corporation. Command and scripting interpreter: Unix shell (T1059.004). MITRE ATT&CK, 2025. https://attack.mitre.org/techniques/T1059/004/.

[23] The MITRE Corporation. Impair defenses (T1562). MITRE ATT&CK, 2025. https://attack.mitre.org/techniques/T1562/.

[24] The MITRE Corporation. MITRE ATT&CK. Knowledge base of adversary tactics and techniques, 2025. https://attack.mitre.org/.

[25] The MITRE Corporation. Network service discovery (T1046). MITRE ATT&CK, 2025. https://attack.mitre.org/techniques/T1046/.

[26] The MITRE Corporation. Scheduled task/job: Cron (T1053.003). MITRE ATT&CK, 2025. https://attack.mitre.org/techniques/T1053/003/.

[27] The MITRE Corporation. Server software component: Web shell (T1505.003). MITRE ATT&CK, 2025. https://attack.mitre.org/techniques/T1505/003/.

[28] Verizon Business. 2025 data breach investigations report. Industry threat report, 2025. https://www.verizon.com/business/resources/reports/2025-dbir-data-breach-investigations-report.pdf.

[29] Wazuh, Inc. Anti-tampering - wazuh agent administration. Wazuh Documentation, 2024. https://documentation.wazuh.com/current/user-manual/agent/agent-management/anti-tampering.html.

[30] Wazuh, Inc. Integration - local configuration (ossec.conf). Wazuh Documentation, 2024. https://documentation.wazuh.com/current/user-manual/reference/ossec-conf/integration.html.

[31] Wazuh, Inc. Rules classification. Wazuh Documentation, 2024. https://documentation.wazuh.com/current/user-manual/ruleset/rules/rules-classification.html.

[32] Wazuh, Inc. Security requirements - wazuh server cluster. Wazuh Documentation, 2024. https://documentation.wazuh.com/current/user-manual/wazuh-server-cluster/security-requirements.html.